if, else, else if Statements

Now it is time to finally start learning how we can program some sort of logic using R! Our first step in this learning journey for programming will be simple if, else, and else if statements.

Here is the syntax for an **if** statement in R:

```
if (condition){
    # Execute some code
}
```

So what does this actually mean if you've never seen an if statement before? It means that we can begin to apply some simple logic to our code. We say *if* some condition is *true* then execute the code inside of the curly brackets.

For example, let's say we have two variables, **hot** and **temp**. Imagine that **hot** starts off as FALSE and temp is some number in degrees. If the **temp** is greater than 80 than we want to assign **hot==TRUE**.

Let's see this in action:

In [3]:

```
hot <- FALSE
temp <- 60
```

In [5]:

```
if (temp > 80){
    hot <- TRUE

}
hot
```

Out[5]:

FALSE

In [10]:

```
# Reset temp
temp <- 100

if (temp > 80){
    hot <- TRUE

}

hot
```

Out[10]:

TRUE

Something to keep in mind is that you should format your code carefully so you can come back later on and easily read it! By convention we align the closing bracket with the **if** statement it refers to. However, because we use brackets we could be sloppy (not good!) and still have the code work out:

In [11]:

```
if( 1 == 1){        print('hi')}
```

```
[1] "hi"
```

In [13]:

```
if(1 ==   1)

{

    print('hi')

              }

# This works...but hard to read!
```

```
[1] "hi"
```

A good editor like RStudio will help you make sure everything is aligned well.

else

If we want to execute another block that occurs if the **if** statement is false, we can use an **else** statement to do this! It has the syntax:

```
if (condition) {
  # Code to execute if true
} else {
  # Code to execute if above was not true
}
```

Notice the alignment of the curly brackets and the use of the *else* keyword. Let's see it in action!

In [14]:

```
temp <- 30

if (temp > 90){
    print("Hot outside!")
} else{
    print("Its not too hot today!")
}
```

```
[1] "Its not too hot today!"
```

else if

What if we wanted more options to print out, rather than just two, the *if* and the *else*? This is where we can use the **else if** statement to add multiple condition checks, using **else** at the end to execute code if none of our conditions match up with and if or else if.

Let's see this in action!

In [15]:

```
temp <- 30

if (temp > 80){
    print("Hot outside!")
} else if(temp<80 & temp>50){
    print('Nice outside!')
} else if(temp <50 & temp > 32){
    print("Its cooler outside!")
} else{
    print("Its really cold outside!")
}
```

```
[1] "Its really cold outside!"
```

In [17]:

```
temp <- 75

if (temp > 80){
    print("Hot outside!")
} else if(temp<80 & temp>50){
    print('Nice outside!')
} else if(temp <50 & temp > 32){
    print("Its cooler outside!")
} else{
    print("Its really cold outside!")
}
```

```
[1] "Nice outside!"
```

Final Example

Let's see a final more elaborate example of **if,else, and else if** to close out our discussion! Let's imagine that we have a store with two items for sale: *ham* and *cheese* and we want an automated report to go to HQ depending on how many we sell:

In [19]:

```
# Items sold that day
ham <- 10
cheese <- 10

# Report to HQ
report <- 'blank'


if(ham >= 10 & cheese >= 10){
    report <- "Strong sales of both items"

}else if(ham == 0 & cheese == 0){
    report <- "Nothing sold!"
}else{
    report <- 'We had some sales'
}
print(report)
```

```
[1] "Strong sales of both items"
```

This is still relatively simple and can be expanded upon, but we just wanted to show how we could use if, else, and else if along with some logical operators. We'll slowly build up more complex tasks!